Exposing Disguises and Tracing Illicit Flows: Dual-View Graph Representation Learning for Money Laundering Detection

Zhong Li, Xinyu Yin, *Student Member, IEEE*, Mingjian Guang, *Member, IEEE*, and Changjun Jiang

***Abstract*—Money laundering is the process of hiding the origin of illicit funds to make them appear legitimate, thereby threatening the integrity of financial systems. To detect money laundering activities, graph neural networks (GNNs) have been widely adopted to model complex relational structures in transaction networks. However, a closer inspection of real-world money laundering cases reveals that launderers deliberately establish connections with multiple licit accounts to mask their illicit attributes. Such disguising behavior introduces network heterogeneity, which contradicts the fundamental assumption of homophily for most GNNs. Additionally, money launderers further conceal their activities by obscuring illicit fund flows through multihop transaction paths. This strategy poses a significant challenge for GNNs, as their limited receptive fields struggle to capture such long-range dependencies. To address these challenges, we propose a dual-view graph representation learning method, named DC-LCG, to detect money laundering. DC-LCG employs complementary local and contextual views to expose disguises and trace illicit flows, respectively. The local view implements a soft-label-guided dynamic grouping and aggregation method that separates nodes into illicit and licit groups, performing probability-weighted aggregations to mitigate network heterogeneity and expose disguises within transaction networks. The contextual view employs a dynamic path pruning method to filter licit nodes and enhance paths relevance, followed by multipath semantic fusion through transformer-based encoding to capture long-range dependencies across multihop transaction paths. A mutual attention mechanism integrates both views to create comprehensive node representations. Experiments on three public transaction datasets show that DC-LCG outperforms state-of-the-art baselines by 2%–10% across evaluation metrics.**

***Index Terms*—Antimoney laundering (AML), attention mechanism, graph neural networks (GNNs), multihop pattern mining.**

Received 11 April 2025; revised 11 September 2025; accepted 11 October 2025. Date of publication 16 January 2026; date of current version 3 April 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62373094 and Grant 62502084; in part by the Natural Science Foundation of Shanghai under Grant 23ZR1401000 and Grant 24YF2701600; in part by the Interdisciplinary Frontier Innovation Team Development Special Fund of Donghua University; in part by Donghua University 2024 Cultivation Project of Discipline Innovation under Grant xkcx-202406; and in part by the Fundamental Research Funds for the Central Universities under Grant 2232025D-46. *(Corresponding author: Mingjian Guang.)*

Zhong Li, Xinyu Yin, and Mingjian Guang are with the School of Information and Intelligent Science, Donghua University, Shanghai 201620, China (e-mail: guangmingjian204@163.com).

Changjun Jiang is with the Department of Computer Science and Engineering, Tongji University, Shanghai 200092, China (e-mail: cjjiang@tongji.edu.cn).

Digital Object Identifier 10.1109/TCSS.2025.3623272

I. Introduction

MONEY laundering refers to the process of disguising illegally obtained funds as legitimate income through systematic financial operations [1]. Such illegal practices involve multiple channels including traditional offshore corporate structures, fictitious trade transactions, and increasingly different forms of digital financial technologies [2], [3]. Money laundering activities not only significantly disrupt financial order but also provide funding channels for other illegal operations, posing substantial threats to socioeconomic security [4], [5]. Consequently, the development of advanced antimoney laundering (AML) technologies has emerged as a critical research domain in safeguarding financial integrity.

Early research on AML predominantly employed rule-based methods with fixed thresholds [6], [7], [8], which criminals could readily circumvent by modifying their transaction patterns. This limitation led researchers to shift toward data-driven machine learning approaches [9], [10], [11], [12], [13], [14], [15] that enhanced suspicious transaction detection by identifying complex associations within financial data. Subsequently, the field incorporated deep sequence learning methods [16], [17], [18], [19], [20], [21], [22] to address the limitations of traditional machine learning methods in processing large volumes of sequential transaction data. However, these methods remained limited in their detection capabilities because they could not effectively represent the relational patterns among transactions, which prevented them from capturing the structural features of financial networks. To address this limitation, researchers implemented graph neural networks (GNNs) [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], which demonstrated superior performance in AML detection through their ability to effectively model topological relationships within transaction networks.

However, existing GNN-based money laundering detection approaches still face two significant challenges.

1) *Camouflaged Money Laundering Nodes:* Money launderers deliberately establish multiple connections with licit accounts to disguise their illicit attributes. This disguised connectivity strategy produces heterogeneous connectivity patterns that deviate from the homophily typically observed in transaction networks. As shown in Fig. 1(a), our analysis of node homophily in the Elliptic dataset [36] reveals significant differences between illicit and

2329-924X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

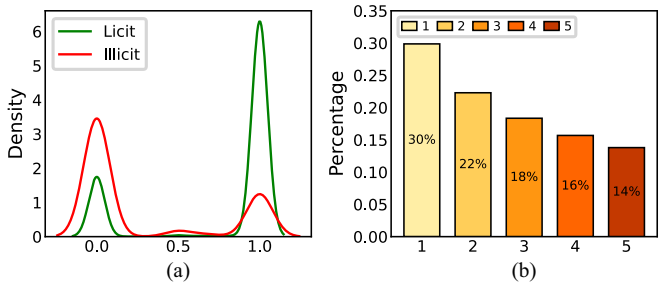


Fig. 1. Analyses of the elliptic dataset: (a) homophily distribution showing neighbor label similarities for licit and illicit nodes; and (b) distribution of shortest illicit transaction path lengths across different radii.

licit nodes. Illicit nodes demonstrate consistently low homophily values approaching 0, which confirms their deliberate disguise within transaction networks. In contrast, licit nodes exhibit high homophily values clustering near 1. This heterogeneous connectivity directly contradicts the fundamental assumption of homophily for most GNN architectures, which presumes that connected nodes tend to share similar features and belong to the same class [35]. Consequently, during neighborhood aggregation in GNNs, this disguising behavior allows the characteristics of licit transactions to dilute the distinctive patterns of illicit transactions, significantly hampering effective detection.

2) *Multihop Money Laundering Paths:* Money launderers transfer illicit funds through multiple intermediate accounts, constructing complex multihop transaction paths. These pathways obscure the flow of funds within transaction networks, rendering the movement of assets difficult to trace. As shown in Fig. 1(b), our analysis of the Elliptic dataset [36] identifies a significant prevalence of multihop propagation in money laundering activities. Quantitative analysis demonstrates that within a transaction hop distance of 5, over 70% of illicit transaction nodes exhibit multihop propagation patterns, indicating that money laundering operations are predominantly conducted through extended transaction paths to evade detection. The prevalence of these extended transaction paths poses a significant challenge for GNN-based methods. Traditional GNN architectures, which typically aggregate information from local neighborhoods with a limited receptive field, struggle to capture the long-range dependencies that are critical in the identification of illicit fund flows [37].

To address these two challenges identified above, we propose a dual-view graph representation learning approach for money laundering detection (DC-LCG). Money laundering operations manifest at two distinct analytical scales: 1) locally through disguised connectivity patterns between individual accounts; and 2) contextually through fund flows traversing multihop transaction paths. Based on this scale differentiation, we design a parallel dual-view architecture comprising a local view and a contextual view. The local view implements a GNN-based soft-label-guided dynamic grouping and aggregation method that uses predicted probabilities to separate neighbors into illicit and licit groups, then applies probability-weighted intragroup aggregation and hybrid attention-based intergroup aggregation to prevent illicit node features from being diluted by deliberately connected licit neighbors. The contextual view generates candidate transaction paths through random walks and then applies a dynamic path pruning method based on soft labels to filter out licit nodes, thereby isolating suspicious transaction paths. Subsequently, multipath semantic fusion processes and aggregates features from these suspicious paths through transformer-based encoding, capturing money laundering patterns across long-range network connections. Furthermore, we implement a mutual attention mechanism that integrates features from both views, creating comprehensive node representations that capture both localized connectivity patterns and multihop transaction contexts.

In summary, our main contributions are as follows.

1) We propose a novel dual-view graph representation learning framework for AML detection that integrates local transaction features with contextual information from transaction paths.
2) In the local view, we design a soft-label-guided dynamic grouping and aggregation method with probability-weighted aggregation and hybrid attention fusion to tackle node camouflaging. Within the contextual view, we introduce a dynamic path pruning method combined with multipath semantic fusion through transformer-based encoding to capture the propagation patterns of illicit fund flows along multihop transaction paths.
3) Experiments on three public transaction datasets demonstrate that our proposed dual-view collaborative framework achieves significant performance improvements over existing methods.

The remainder of this article is organized as follows. We review related work in Section II. We present the problem descriptions in Section III. We provide a detailed explanation of the various modules in the DC-LCG framework in Section IV. We report experiments and performance analyses in Section V. Finally, we conclude this article in Section VI.

II. Related Works

We summarize related studies of AML in three main categories as follows: 1) traditional machine learning-based AML methods; 2) sequence-based AML methods; and 3) GNN-based AML methods.

A. Traditional Machine Learning-Based AML Methods

Traditional machine learning approaches [9], [10], [11], [12], [13], [14], [15] have been applied to AML detection. For example, Tang et al. proposed a support vector machine model with a customized radial basis function kernel using heterogeneous value difference metric for AML [9]. Lv et al. developed a neural network utilizing adaptive pattern clustering and recursive least square algorithms for detecting suspicious financial transactions [10]. Gao et al. proposed a cluster-based local outlier factor algorithm combining distance-based clustering and local

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

outlier detection for AML [11]. Jayasree and Balan designed a bitmap index-based decision tree technique for evaluating money laundering regulatory risk in financial institutions [12]. Harlev et al. developed a supervised machine learning approach using gradient boosting to deanonymize entity types on the bitcoin blockchain [13]. Hu et al. developed classifiers using graph features such as immediate neighbors, curated features, and node2vec embeddings to detect money laundering transactions [14]. Kumar et al. proposed using naive Bayes classification on customized datasets to identify money laundering transactions [15]. However, these traditional machine learning approaches treat transactions as isolated events, failing to capture the graph structure and temporal dynamics inherent in real-world transaction networks.

B. Sequence-Based AML Methods

In recent years, deep sequence learning [16], [17], [18], [19], [20], [21], [22] has been increasingly applied to AML detection. For example, Zhao et al. developed an attention-based long short-term memory (LSTM) using transaction features to classify transaction activities as ponzi schemes, other scams, or legitimate transactions [16]. Sun et al. proposed a supervised learning approach that extracts transaction trees from the bitcoin blockchain and classifies them using a multilayer LSTM to detect coin mixing transactions for AML [17]. Rodríguez et al. developed a financial fraud detection system that identifies unusual spending patterns by analyzing historical transactions with attention mechanisms and unsupervised multitask learning [18]. Umer et al. built an ensemble model merging convolutional neural and LSTM to identify fraud on the Ethereum blockchain [19]. Tatulli et al. developed a hierarchical transformer that examines transactions at both individual and sequence levels to detect money laundering [20]. Huang et al. proposed a model that combines an intensity-aware transformer to detect subtle behavior changes and sequence-based contrastive learning to address label scarcity for AML [21]. Girish and Bhowmik created an approach using recurrent neural networks and hybrid ensembles for money laundering detection in banking transactions [22].

However, the above sequence-based AML detection methods have several key limitations. First, many of these methods focus only on detecting anomalies in individual transactions. Second, some are limited to identifying relatively narrow, predefined laundering structures, potentially missing novel or adaptive strategies. Finally, most approaches fail to fully leverage the rich structural information in transaction networks by not analyzing the data from a graph perspective, limiting their ability to detect sophisticated money laundering patterns.

C. GNN-Based AML Methods

GNNs have emerged as a promising approach for money laundering detection, building upon classical architectures such as graph convolutional network (GCN) [23], graph attention network (GAT) [24], and GraphSAGE [25]. These architectures effectively capture complex relationships between entities in transaction networks, enabling the identification of suspicious patterns that might indicate money laundering activities. For example, Liu et al. developed a GNN that addresses class imbalance in fraud detection by sampling nodes based on label distribution and separately selecting neighbors for minority and majority classes [27]. Yu et al. developed an aspect-level GNN that encodes AML risk from multiple perspectives and uses attention mechanisms to identify illicit transactions [28]. Chai et al. introduced a framework using supervised contrastive learning to discover money laundering subnetworks in transaction data for AML [29]. Hyun et al. proposed a multirelational GNN that leverages cooperative edges to detect money laundering tactics such as mixing services and uses dual-feature aggregation to capture both homophilic and heterophilic information [30]. Li et al. developed a semisupervised diffusion model that identifies suspicious transaction patterns, detects anomalous subgraphs, and generates explanatory heat maps for AML [31]. Jing et al. introduced a model which separates homophilic and heterophilic features in node embedding to represent transaction network entities and applies oversampling to handle imbalanced AML data [32]. Karim et al. proposed combining node embeddings from transaction graphs with graph-based features to train ensemble classifiers for detecting money laundering activities [33]. Luo et al. developed a semisupervised framework utilizing a GNN to learn node representations from a similarity-weighted transaction network and a contrastive self-supervised learning model in a decoupled pretraining phase for money laundering detection [34].

However, the existing GNN-based AML detection methods have two key limitations. Many methods employ fixed neighborhood sampling or embedding strategies that fail to dynamically adapt to the evolving nature of money laundering tactics. Additionally, these approaches lack fine-grained aggregation mechanisms to differentiate between illicit and licit neighbors, reducing effectiveness in detecting camouflaged money laundering where fraudulent nodes connect with legitimate accounts.

III. Preliminaries

In this section, we present essential preliminaries, including the definition of a transaction graph and the problem statement.

A. Definition

Definition 1: A transaction graph is denoted as $G = \{V, E\}$ with node features X, adjacency matrix A, and node labels Y where $V = \{1, 2, \ldots, N\}$ represents the nodes, each corresponding to a transaction, and $E \subseteq V \times V$ represents the set of edges between nodes. Each node $i \in V$ is associated with a feature vector $x_i \in \mathbb{R}^S$, where S denotes the number of features per transaction. These vectors collectively form the feature matrix $X \in \mathbb{R}^{N \times S}$. An edge $(i, j) \in E$ exists when the receiving account of transaction i is the sending account of transaction j, representing the flow of funds between transactions. The adjacency matrix $A \in \mathbb{R}^{N \times N}$ encodes this graph structure, where $A_{i,j} = 1$ if there is an edge between vertices i and j, and $A_{i,j} = 0$ if no edge exists between them. Additionally, each node i is assigned a label vector $y_i \in Y$ indicating whether the

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

transaction is licit or illicit, where y_i represents the one-hot encoded vector of the true class.

B. Problem Statement

Our research focuses on developing effective methods for identifying money laundering activities in financial transaction networks. The detection task is formulated as a binary node classification problem on a transaction graph, where the objective is to accurately classify each transaction node as illicit when $y_i = [0, 1]$ or licit when $y_i = [1, 0]$. With the notation established in Definition 1, we formally define our problem as follows.

Input: Given a transaction graph $G = \{V, E\}$ with node features X and its label information Y as supervision.

Output: For classification task, our goal is to predict the node labels using our proposed DC-LCG. For representation fusion, we utilize both the local view representation, which models illicit camouflaging behavior, and the contextual view representation, which encodes the multihop propagation patterns of illicit funds. These representations serve as inputs to a mutual attention mechanism to generate the final comprehensive representation.

IV. Methodology

In this section, we introduce our proposed AML detection model, DC-LCG. We first preprocess the transaction data using a dynamic value field encoder to enrich the raw features in Section IV-A. Then, in the local view, a GNN-based dynamic grouping and aggregation method is employed to mitigate heterogeneity and effectively eliminate money laundering disguises in Section IV-B. Subsequently, within the contextual view, a dynamic path pruning method and a transformer-based encoder with fusion capabilities are applied to capture the propagation patterns of illicit fund flows along multihop transaction paths in Section IV-C. Finally, a mutual attention feature fusion module designed to integrate information from both the local and contextual views, along with a classification component, is presented in Section IV-D. The conceptual framework of our proposed model is displayed in Fig. 2.

A. Dynamic Value Field Encoder

Transaction monitoring reveals that specific ranges of transaction attributes correlate strongly with suspicious or legitimate behaviors. For instance, specific time periods and transaction amount ranges frequently appear in money laundering cases, while others typically indicate legitimate transactions. These interval-behavior relationships contain discriminative information for identifying illegal financial activities that cannot be identified by examining individual attribute values in isolation.

To capture these associations between attribute value ranges and transaction behaviors, we introduce a tree-based dynamic value field encoder. This encoder leverages the information gain criterion to automatically partition continuous attribute values into optimal discriminative intervals. It then represents these intervals using one-hot encoding, forming enhanced features and thereby preserving the interval-based discriminative information throughout the feature extraction process. As illustrated in Fig. 2(a), the dynamic value field encoder enhances the node features from X to $\tilde{X}$, transforming the original graph representation $G(X)$ into the enhanced graph representation $G(\tilde{X})$.

The dynamic value field encoder consists of two sequential operations: 1) interval partitioning followed by encoding; and 2) concatenation. Initially, decision trees are employed to segment continuous attribute domains into discriminative intervals. Formally, the original feature vector for center node i is denoted as $x_i = [x_i[1], x_i[2], \ldots, x_i[S]]$, where each $x_i[s]$ represents the value of the sth attribute. To determine the optimal partitions, decision trees are employed to segment the domain of each attribute into J_s mutually exclusive intervals. We denote this set of intervals as $\{I_r\}_{r=1}^{J_s}$, where I_r represents the rth interval.

Subsequently, the encoder applies one-hot encoding to transform interval assignments into vector representations. Each attribute $x_i[s]$ is transformed through the one-hot encoding method into a segmented feature vector $\tilde{x}_i'[s]$ of length J_s. The rth element $\tilde{x}_i'[s][r]$ is determined as computed

$$\tilde{x}_i'[s][r] = \begin{cases} 1, & \text{if } x_i[s] \in I_r \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

After transforming each attribute individually, we obtain the complete segmented feature for node i by concatenating these segmented features, resulting in $\tilde{x}_i' = [\tilde{x}_i'[1], \tilde{x}_i'[2], \ldots, \tilde{x}_i'[S]]$. To preserve both the original feature properties and the discriminative properties derived from interval partitioning, the original feature vector x_i is combined with its segmented counterpart $\tilde{x}_i'$ to form the enhanced feature $\tilde{x}_i = [x_i, \tilde{x}_i']$ for node i. The procedure of the dynamic value field encoder is shown in Algorithm 1.

The resulting enhanced features effectively capture both continuous attribute information and discrete interval associations with transaction behaviors, thereby enhancing the model's capability to identify illicit financial activities. This feature enhancement process is carried out as a preprocessing step, ensuring computational efficiency. It is not repeated during subsequent training epochs, thereby maintaining the model's ability to capture range-behavior correlations without incurring additional computational cost.

B. Dynamic Grouping and Aggregation

In this section, we introduce a soft-label-guided dynamic grouping and aggregation method to address the challenge of illicit transactions deliberately establishing connections with numerous licit transactions to mask their suspicious patterns. Our approach employs dynamic soft-label-guided grouping to separate neighbors into distinct categories, followed by probability-weighted intragroup aggregation and hybrid attention-based intergroup fusion to mitigate network heterogeneity. As illustrated in Fig. 2(b), our approach consists of three principal components: 1) dynamic grouping; 2) intralayer aggregation; and 3) interlayer aggregation.

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

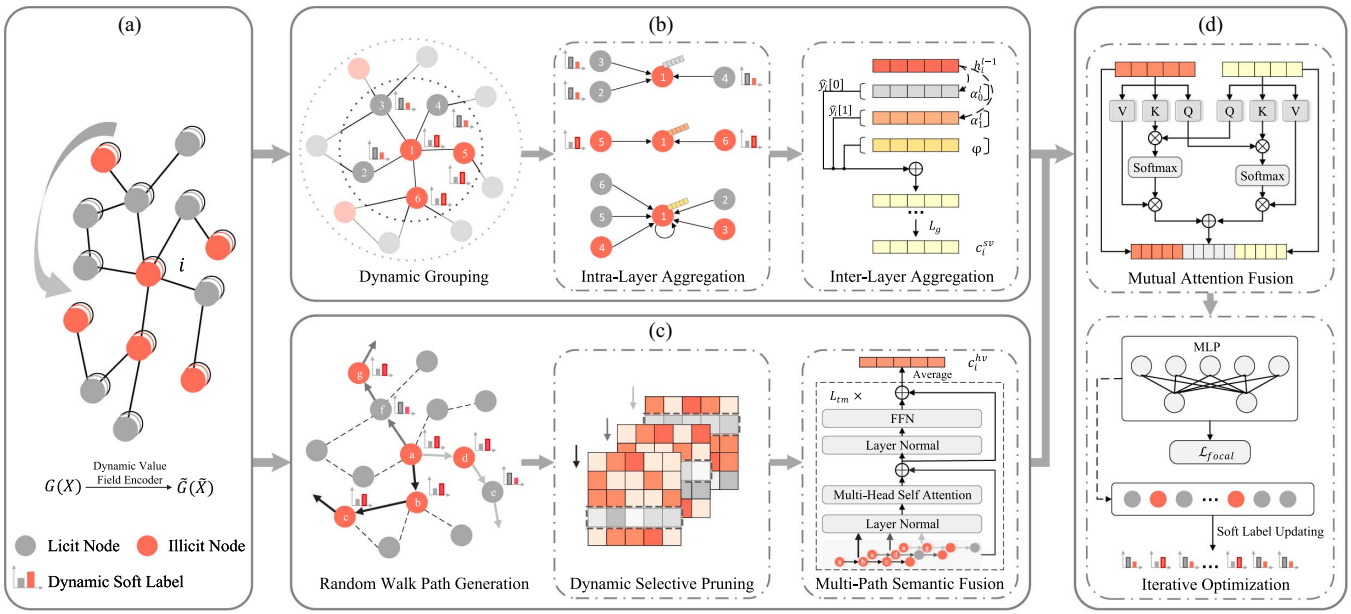


Fig. 2. Overall architecture of DC-LCG comprises four principal modules: (a) dynamic value field encoder; (b) dynamic grouping and aggregation that consists of node embedding initialization, intralayer aggregation, and interlayer aggregation; (c) dynamic path pruning and fusion that includes random walk path generation, dynamic selective pruning, and multipath semantic fusion; and (d) fusion and optimization that integrates mutual attention fusion and iterative optimization with soft label updating for illicit node detection.

To establish a foundational representation space for subsequent analysis, we transform the initial node features through a multilayer perceptron (MLP). The resulting embedding is formulated as $H^{(0)} = \mathrm{MLP}(\tilde{X})$, where $\tilde{X}$ represents the enhanced feature matrix. At layer l, the node representations are characterized by $H^{(l)} = \{h_1^{(l)}, h_2^{(l)}, \ldots, h_N^{(l)}\}$, with the final representation after L_g iterations denoted as H^{L_g}.

At each layer, during the propagation from layer $l-1$ to layer l, our approach implements dynamic grouping to differentiate between licit and illicit nodes among neighbors, thereby improving the distinguishability of grouped messages. Specifically, We denote $\boldsymbol{\mathcal{N}}(i) = \{j \mid A_{i,j} = 1, j \neq i\}$ as the neighbor set of center node i. The dynamic grouping leverages node category predictions from the previous training epoch. For each neighbor node j of i, the prediction result is represented as a vector $\hat{y}_j = [\hat{y}_j[0], \hat{y}_j[1]]$, where $\hat{y}_j[0]$ indicates the probability of node j being classified as licit, and $\hat{y}_j[1]$ represents its probability of being illicit. The grouping of neighbor nodes is defined as

$$\boldsymbol{\mathcal{N}}_g(i) = \{j \mid j \in \boldsymbol{\mathcal{N}}(i), \hat{y}_j[g] > \hat{y}_j[1-g]\} \tag{2}$$

where $\boldsymbol{\mathcal{N}}_g(i)$ represents the neighbor subset of node i assigned to the group g, with $g = 0$ and $g = 1$ representing the licit and illicit groups, respectively. Under this classification, when $g = 0$, node j belongs to the licit group, whereas when $g = 1$, node j belongs to the illicit group.

Following grouping, to minimize feature interference between different node groups, we employ a hierarchical aggregation strategy consisting of two stages: 1) intragroup aggregation; and 2) intergroup aggregation. By distinguishing between these two aggregation processes, our method preserves the distinctive features of illicit nodes, ensuring they are not overshadowed by those of licit nodes.

Algorithm 1: Dynamic Value Field Encoder.

Input: Feature matrix X, labels Y, feature dimension S, number of intervals J_s for each feature
Output: Enhanced feature matrix $\tilde{X}$

```
Initialize segmented feature matrix X̃′;
for s = 1 to S do
    Build a Decision Tree (DT) based on (X[:, s], Y);
    Extract all split values as split_list from DT;
    Sort split_list and divide into J_s intervals;
    Map values in X[:, s] to their interval indices 0 to J_s − 1;
    One-hot encode these interval indices to get X̃′[:, s] by (1);
    Append X̃′[:, s] to X̃′;
end
X̃ = [X, X̃′];
return X̃;
```

The intragroup aggregation process comprises three sequential computational phases. The initial phase performs global aggregation on node representations from all neighbor nodes, establishing a comprehensive feature baseline. The subsequent phase applies specialized aggregation functions independently to the licit and illicit node groups, thereby preserving their distinctive characteristics. The global aggregation operation across all neighbor nodes is formulated as

$$h_{i,\mathrm{all}}^{'(l)} = \mathrm{Agg}_{\mathrm{mean}}\left(h_j^{(l-1)} \mid j \in \boldsymbol{\mathcal{N}}(i)\right) \tag{3}$$

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

$$h_{i,\text{all}}^{(l)} = \text{ReLU}\left(W_{\text{intra-all}}^{(l)}\left(h_i^{(l-1)}, h_{i,\text{all}}^{'(l)}\right)\right) \tag{4}$$

where $h_{i,\text{all}}^{(l)}$ represents the aggregated feature from the neighbors of node i, $h_i^{(l-1)}$ and $h_j^{(l-1)}$ represent the feature vectors of node i and node j at the previous layer $l-1$, respectively, $\text{Agg}_{\text{mean}}(\cdot)$ denotes the average aggregation operation, $W_{\text{intra-all}}^{(l)}$ is the weight matrix for neighbor information aggregation, and $\text{ReLU}(\cdot)$ is the ReLU activation function.

During the specialized intragroup aggregation phases, each node's feature within its group is dynamically assigned weight based on soft label probabilities, to extract consistent features among nodes in the same group. The formulation is expressed as

$$h_{i,g}^{'(l)} = \text{Agg}_{\text{mean}}\left(\hat{y}_j[g] h_j^{(l-1)} \mid j \in \boldsymbol{\mathcal{N}}_g^{(l)}(i)\right) \tag{5}$$

$$h_{i,g}^{(l)} = \text{ReLU}\left(W_{\text{intra-g}}^{(l)} h_{i,g}^{'(l)}\right) \tag{6}$$

where $h_{i,g}^{(l)}$ represents the aggregated features from neighbor nodes in the group g, $\hat{y}_j[g]$ is the soft label probability of node j belonging to the group g, which determines the weight of the node in aggregation process, and $W_{\text{intra-g}}^{(l)}$ is the trainable weight matrix for the group-specific feature transformation. This probability-weighted intragroup aggregation ensures that nodes with higher membership confidence contribute more significantly to their group's representation, preventing feature dilution from misclassified neighbors.

The intergroup aggregation subsequently combines representations from different node groups while preserving the discriminative characteristics of each group. To achieve this objective, we implement a hybrid weighting scheme that applies attention weights combined with soft label probabilities to features from different groups. This allows the center node to both capture its attention strength with different groups through the attention mechanism and quantify its confidence levels toward various group assignments using soft label probabilities. The aggregated feature $h_i^{(l)}$ for the center node i at layer l is formulated as

$$h_i^{(l)} = \sum_g \alpha_g^{(l)} \hat{y}_i[g] h_{i,g}^{(l)} + \varphi h_{i,\text{all}}^{(l)} \tag{7}$$

$$\alpha_g^{(l)} = \frac{\exp\left(\omega_g^{(l)}\right)}{\sum_{g\in\{0,1\}} \exp\left(\omega_g^{(l)}\right)} \tag{8}$$

$$\omega_g^{(l)} = q^\top \cdot \tanh\left(W_{\text{inter1}}^{(l)} h_i^{(l-1)} + W_{\text{inter2}}^{(l)} h_{i,g}^{(l)}\right) \tag{9}$$

where $W_{\text{inter1}}^{(l)}$ and $W_{\text{inter2}}^{(l)}$ are weight matrices for interlevel aggregation, q is a learnable parameter vector determining the attention parameter $\alpha_g^{(l)}$ for group g, $\top$ denotes the transpose operation, and φ is a learnable scalar controlling the weight proportion of overall neighbor aggregated features $h_{i,\text{all}}^{(l)}$ in the final node embedding. This hybrid weighting mechanism enables the model to adaptively balance attention-based importance with confidence-based group membership, ensuring that heterogeneous connectivity patterns do not overwhelm the distinctive characteristics that identify illicit transactions.

After L_g iterations, the final embedding for node i is obtained as $c_i^{sv} = h_i^{(L_g)}$, where c_i^{sv} represents the final local representation of node i extracted by this module.

C. Dynamic Path Pruning and Fusion

In this section, we introduce a dynamic path pruning and fusion method to trace illicit financial flows across transaction networks, leveraging a contextual perspective. Our approach employs similarity-based dynamic pruning to isolate suspicious transaction sequences, followed by multipath semantic fusion that captures long-range dependencies through transformer-based encoding and contextual aggregation across multiple transaction paths. As illustrated in Fig. 2(c), our approach comprises three integrated components: 1) random walk path generation; 2) dynamic selective pruning; and 3) multipath semantic fusion.

The initial stage generates potential fund flow trajectories through random walk sampling. Specifically, starting from a central node i, we generate M random walk paths of length $(T+1)$ based on the graph's topological structure. This results in a candidate path set $\boldsymbol{\mathcal{P}}_i = \{P_{i,m}\}_{m=1}^M$, where $P_{i,m}$ denotes the mth random walk path originating from node i. Each path $P_{i,m}$ is represented as a sequence $\{\tilde{x}_i^0, \tilde{x}_{i,m}^1, \tilde{x}_{i,m}^2, \ldots, \tilde{x}_{i,m}^T\}$, where $\tilde{x}_i^0$ represents the enhanced feature of the central node i, and $\tilde{x}_{i,m}^t$ denotes the enhanced feature of the node at the tth hop along $P_{i,m}$.

Following path generation, we apply dynamic selective pruning to filter irrelevant connections and retain meaningful transaction sequences. This pruning operation requires assessing the behavioral consistency between nodes along each path and the central node. To evaluate the relevance of each node in the path with respect to the central node i, we introduce a soft-label-based similarity metric. For the node at hop t with features $\tilde{x}_{i,m}^t$ in path $P_{i,m}$, its similarity to the central node is defined as

$$\text{Sim}_{\text{label}}(\tilde{x}_{i,m}^t, \tilde{x}_i^0) = \frac{\hat{y}_{\tilde{x}_i^0} \hat{y}_{\tilde{x}_{i,m}^t}}{\|\hat{y}_{\tilde{x}_i^0}\| \|\hat{y}_{\tilde{x}_{i,m}^t}\|} \tag{10}$$

where $\hat{y}_{\tilde{x}_i^0}$ and $\hat{y}_{\tilde{x}_{i,m}^t}$ are the dynamic soft-label vectors of the central node and tth node, respectively, and $\|\cdot\|$ represents the L_2-norm.

For each candidate path $P_{i,m}$, we rank all nodes except the central node by their similarity scores in descending order. Based on this ranking, we perform a selective pruning process to retain only the most relevant nodes. To control the pruning process, we introduce a dropout ratio β, where $0 \le \beta < 1$. Specifically, we retain only the top-ranked T' nodes, where $T' = \lceil T(1-\beta) \rceil$, forming a pruned path $P'_{i,m} = \{\tilde{x}_i^0, \tilde{x}_{i,m}^{1'}, \tilde{x}_{i,m}^{2'}, \ldots, \tilde{x}_{i,m}^{T'}\}$, where $\tilde{x}_{i,m}^{t'}$ represents the feature of the node at the t'th hop in the pruned path $P'_{i,m}$. This soft-label-based similarity metric evaluates behavioral consistency rather than structural proximity, enabling the identification of nodes that exhibit similar risk characteristics to the central node along multihop transaction sequences.

After applying the selective pruning mechanism to all candidate paths, we establish the pruned path set $\boldsymbol{\mathcal{P}}'_i = \{P'_{i,m}\}_{m=1}^M$.

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

To incorporate contextual information from these pruned paths, we implement a multipath semantic fusion approach utilizing a transformer-based encoder. Specifically, we employ a transformer architecture to encode representations across the filtered multiple paths, subsequently aggregating the central node's features to derive comprehensive contextual node representations.

For each path $P'_{i,m}$ in the updated path set $\boldsymbol{\mathcal{P}}'_i$, we generate sequential token representations through position-aware linear feature mapping. We employ distinct transformation matrices for nodes at different hop distances, acknowledging the varying significance of positional information within path structures. The sequential representation is formulated as

$$Z_{i,m}^{(0)} = \left[\tilde{x}_i^0 W_0, \tilde{x}_{i,m}^{1'} W_{1'}, \tilde{x}_{i,m}^{2'} W_{2'}, \ldots, \tilde{x}_{i,m}^{T'} W_{T'}\right] \tag{11}$$

where $W_{t'}$ denotes the learnable mapping matrix corresponding to the t'th hop position. This position-specific parameterization enables the model to differentiate and process feature information contextually based on the topological distance from the central node.

The resulting sequence $Z_{i,m}^{(0)}$ is then fed into a transformer encoder comprising L_{tm} layers to extract semantic features. Each layer consists of a multihead self-attention (MHA) and a feed-forward network (FFN), both incorporating a residual connection and a layer normalization prior to the sublayer computations. For the lth layer, the previous layer's output $Z_{i,m}^{(l-1)}$ is first passed through a layer normalization function, denoted as $\tilde{Z}_{i,m}^{(l-1)} = \mathrm{LN}\left(Z_{i,m}^{(l-1)}\right)$, before being further processed.

The normalized representation is subsequently processed by the MHA. Specifically, $\tilde{Z}_{i,m}^{(l-1)}$ is projected onto three distinct spaces using layer-l-specific and attention head-h-specific weight matrices $W_Q^{(l,\mathrm{h})}$, $W_K^{(l,\mathrm{h})}$, and $W_V^{(l,\mathrm{h})}$ to obtain query, key, and value matrices $Q_{i,m}^{(l,\mathrm{h})}$, $K_{i,m}^{(l,\mathrm{h})}$, and $V_{i,m}^{(l,\mathrm{h})}$. The attention computation is defined as

$$\mathrm{head}_{i,m}^{(l,\mathrm{h})} = \mathrm{Softmax}\left(\frac{Q_{i,m}^{(l,\mathrm{h})}\left(K_{i,m}^{(l,\mathrm{h})}\right)^\top}{\sqrt{d_k}}\right) V_{i,m}^{(l,\mathrm{h})} \tag{12}$$

$$\mathrm{MHA}\left(\tilde{Z}_{i,m}^{(l-1)}\right) = \left(\mathrm{head}_{i,m}^{(l,1)}, \cdots, \mathrm{head}_{i,m}^{(l,\mathrm{H})}\right) W_O^{(l)} \tag{13}$$

where $\mathrm{head}_{i,m}^{(l,\mathrm{h})}$ is the output of this attention mechanism, d_k is the dimension of the key vectors, $\mathrm{Softmax}(\cdot)$ is a function that normalizes the attention scores into a probability distribution, and $W_O^{(l)}$ is the linear transformation matrix for the lth layer.

The output of the MHA is further processed using residual connections, the LN, and the FFN. The formula is as follows:

$$Z_{i,m}^{'(l)} = \mathrm{MHA}\left(\tilde{Z}_{i,m}^{(l-1)}\right) + \tilde{Z}_{i,m}^{(l-1)} \tag{14}$$

$$Z_{i,m}^{(l)} = \mathrm{FFN}\left(\mathrm{LN}\left(Z_{i,m}^{'(l)}\right)\right) + Z_{i,m}^{'(l)}. \tag{15}$$

Through L_{tm} layers of stacking, the path $\boldsymbol{\mathcal{P}}'_i$ set obtains its final sequential token sequences $\left\{Z_{i,1}^{(L_{\mathrm{tm}})}, Z_{i,2}^{(L_{\mathrm{tm}})}, \ldots, Z_{i,M}^{(L_{\mathrm{tm}})}\right\}$, where $Z_{i,m}^{(L_{\mathrm{tm}})}$ is the final token sequence for subpath $P'_{i,m}$, used for subsequent feature aggregation.

To obtain a contextual feature of the node i, the encoded representations corresponding to the central node from different paths are aggregated. Specifically, the contextual feature c_i^{hv} is computed as the average of the representations $Z_{i,m}^{(L_{\mathrm{tm}})}$ at index [0] from each path, as expressed by the formula $c_i^{hv} = (1/M)\sum_{m=1}^{M} Z_{i,m}^{(L_{\mathrm{tm}})}[0]$, where $Z_{i,m}^{(L_{\mathrm{tm}})}[0]$ denotes the representation of the first token corresponding to the central node from the mth path after L_{tm} encoding layers. This mean-based aggregation mechanism enables the integration of diverse contextual information from multiple paths into the comprehensive contextual representation c_i^{hv} of node i.

D. Fusion and Optimization

In this section, we strive to accomplish two primary objectives: 1) the integration of local with contextual representations; and 2) the development of an iterative optimization model. As depicted in Fig. 2(d), this section is composed of two distinct components: mutual attention fusion and iterative optimization.

To achieve the integration of local and contextual representations, we propose a bidirectional mutual attention mechanism to capture dependencies between local and contextual information. This mechanism establishes a comprehensive representation space that leverages complementary information from both analytical perspectives. Initially, local and contextual representations c_i^{sv} and c_i^{hv} are transformed into corresponding query, key, and value vectors through learnable weight matrices. For the local representation, these transformations yield q_i^{sv}, k_i^{sv}, and v_i^{sv}. Similarly, the contextual representation is mapped to q_i^{hv}, k_i^{hv}, and v_i^{hv}.

Next, local and contextual representations serve as queries to attend to each other's key–value pairs. For example, when the local representation attends to the contextual representation, the attention output is computed as follows:

$$c_i^{\mathrm{sv}\to\mathrm{hv}} = \mathrm{softmax}\left(\frac{q_i^{\mathrm{sv}}\left(k_i^{\mathrm{hv}}\right)^\top}{\sqrt{d_k}}\right)\cdot v_i^{\mathrm{hv}} \tag{16}$$

where $c_i^{\mathrm{sv}\to\mathrm{hv}}$ represents the information flow from the contextual representation to local representation, capturing how contextual features influence and enhance the local representation. Similarly, the attention output $c_i^{\mathrm{hv}\to\mathrm{sv}}$ is obtained when the contextual representation attends to the local representation.

The final fused representation c_i is obtained by aggregating the attention outputs with the original representations. The aggregated attention output is computed as follows:

$$c_i^{\mathrm{attn}} = \mathrm{Agg}_{\mathrm{mean}}\left(c_i^{\mathrm{sv}\to\mathrm{hv}}, c_i^{\mathrm{hv}\to\mathrm{sv}}\right) \tag{17}$$

where c_i^{attn} is the aggregated attention output combining the two attended representations. The final fused representation $c_i = \left(c_i^{\mathrm{sv}}, c_i^{\mathrm{attn}}, c_i^{\mathrm{hv}}\right)$ is obtained by concatenating the original vector representation, the aggregated attention output, and the hidden vector representation. This bidirectional attention enables enhanced detection by revealing cases where nodes exhibit legitimate local patterns but participate in suspicious multihop sequences, and conversely, identifying nodes that appear normal in extended paths but demonstrate camouflaging behaviors in their immediate neighborhoods.

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

Using these comprehensive representations that integrate both local and contextual money laundering features, we subsequently perform node-level binary classification by feeding the fused representation c_i into a MLP and computing the predicted probability distribution $\hat{y}_i$ via the softmax function. To optimize the model, we employ focal loss as the loss function. The complete training process is defined as

$$\hat{y}_i = \text{Softmax}(\text{MLP}(c_i)) \tag{18}$$

$$\mathcal{L} = -\sum_{i \in V} y_i \cdot \alpha \cdot (1 - \hat{y}_i)^{\gamma} \cdot \log(\hat{y}_i) \tag{19}$$

where y_i represents the one-hot encoded classification vector for node i, $\hat{y}_i$ is the predicted probability vector of node i, γ is a modulating factor used to enhance the impact of hard-to-classify samples, and α is a balancing factor to address the imbalance between positive and negative samples.

After each training iteration, we dynamically update the predicted probabilities $\hat{y}_i$ as new pseudolabels and use them to guide subsequent training. This soft-label-based strategy preserves the interclass probability distribution, thereby capturing more fine-grained classification information. By iteratively updating, the pseudolabels based on the model's predictions and using them to guide the training process in subsequent iterations, we establish a positive feedback loop that progressively refines the model's discriminative power and generalization ability. The detailed learning procedure of DC-LCG is shown in Algorithm 2.

Algorithm 2: The Detailed Learning Procedure of DC-LCG.

Input: Maximum epochs E_{epoch}, Graph G , feature matrix X, adjacency matrix A, labels Y, GNN layers L_g, transformer layers L_{tm}

Output: Categorical output $\hat{Y}$ of the graph G

```
Compute X̃ by fitting value field encoder with (X, Y);
Initialize soft label buffer;
for e = 1 to E_epoch do
    for l = 0 to L_g do
        Group neighbor nodes using soft label buffer by (2);
        Apply inter-group aggregation by (3)–(6);
        Update node features h^l with hybrid weighting by (7)–(9);
    end
    Extract local node embeddings C^sv from final GNN layer;
    Generate path sets P by random walk sampling;
    Prune paths to P' by (10);
    Obtain contextual node embeddings C^hv by encoding path sequences through L_tm-layer transformer encoder by (11)–(15);
    Execute mutual attention fusion between C^sv and C^hv to generate fused representation C by (16)–(17);
    Update the total loss by (18)–(19);
    Use back-propagation to update model parameters;
    Update soft label buffer;
end
return The classification result of Graph G;
```

V. Experiments

In this section, we evaluate the effectiveness of our proposed DC-LCG model through comprehensive experiments. We introduce datasets in Section V-A. Then, we present baseline methods selected for performance comparison in Section V-B. Evaluation metrics and implementation details of our approach are described in Section V-C. Subsequently, we demonstrate the superior performance of DC-LCG against state-of-the-art methods through quantitative experiments in Section V-D. Furthermore, we conduct parameter evaluation to analyze the impact of key hyperparameters on model performance in Section V-E. We perform ablation studies to validate the effectiveness of key components in our proposed model in Section V-F. Additionally, we analyze the computational complexity and runtime performance of our approach in Section V-G. Finally, we present visualizations of money laundering detection cases in Section V-H.

A. Datasets

In this section, we present the three public datasets used in our experiments. The characteristics of each dataset are detailed in Table I.

1) *Elliptic [36]:* The Elliptic dataset consists of 49 temporal Bitcoin transaction networks, each representing a distinct time step, specifically designed for money laundering detection research. Within these networks, nodes represent individual bitcoin transactions, with edges denoting the flow of cryptocurrency between transactions. The dataset contains both licit transactions from exchanges, wallet providers, and miners and illicit transactions associated with scams, malware, and other criminal activities. We employ this dataset for identifying illicit nodes within Bitcoin transaction networks.
2) *RD [38]:* The RD dataset extends the Elliptic dataset by incorporating 19 additional features derived from Bitcoin deanonymization hash functions. These supplementary features are derived through a bitcoin data scraper. To simulate real-world scenarios with incomplete data, the RD dataset implements varying degrees of data sparsity (5%, 10%, 15%, and 20% missing rates), reflecting practical data limitations. We utilize the RD dataset with 20% missing data for illicit node detection.
3) *T-Finance [39]:* The T-Finance dataset is specifically designed for detecting anomalous accounts in transaction networks. In this dataset, nodes correspond to anonymized account entities, while edges represent inter-account transactions. Node labels are derived from behavioral patterns associated with illicit activities, including financial fraud, money laundering operations, and unauthorized gambling transactions. We utilize this dataset

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

TABLE I
STATISTICS OF THREE DATASETS

Dataset	Nodes	Edges	Features	Illicit(%)
Elliptic	46 564	73 248	166	9.76
RD	46 564	73 248	185	9.76
T-Finance	39357	21 222 543	10	4.58

to evaluate the model's capacity to identify anomalous accounts within financial networks.

B. Baselines for Comparison

To examine the efficacy of our proposed approach, we select several baseline models for comparative experiments. These models span a range of computational paradigms, from traditional machine learning algorithms to specialized neural network architectures. The baseline models are outlined as follows.

1) *MLP [40]:* A feedforward artificial neural network consisting of multiple layers of perceptrons with nonlinear activation functions.
2) *XGBoost [41]:* A machine learning algorithm that implements gradient boosting within a scalable, distributed framework, leveraging regularization techniques to mitigate overfitting and optimize computational resources.
3) *RF [42]:* An ensemble methodology that constructs multiple decision trees during training and outputs the mode of their individual classifications.
4) *GCN [23]:* A neural network variant that performs convolution operations directly on graph structures by aggregating node representations across neighborhoods.
5) *Skip-GCN [36]:* An architectural modification to traditional GCNs that implements residual connections between nonconsecutive layers to address the oversmoothing problem.
6) *GraphSAGE [25]:* A neighborhood aggregation framework that generates embeddings by sampling and aggregating features from a node's local neighborhood.
7) *GAT [24]:* A GNN that employs attention mechanisms to assign weights to neighbor nodes during feature aggregation.
8) *PC-GNN [27]:* A fraud-detection specialized architecture that implements a neighbor selection mechanism to distinguish between different node types during message-passing.
9) *Care-GNN [43]:* A heterophily-aware GNN that employs relation-specific aggregation to incorporate information from neighbors through an iterative procedure.
10) *MLaD2 [34]:* A semisupervised money laundering detection framework that employs decoupling training with separate pre-training and fine-tuning phases, constructing a transaction relationship network based on node similarity to overcome the homogeneity hypothesis limitation in traditional GNN approaches.
11) *DGT + XGBoost [33]:* A hybrid pipeline approach that combines dynamic graph transformer (DGT) embeddings with XGBoost classification, where DGT captures both spatial and temporal graph dynamics through attention mechanisms before feeding the learned representations to gradient boosting for final prediction.

To systematically evaluate our proposed DC-LCG approach, we categorize the baseline methods into three distinct groups based on their architectural characteristics. The first group encompasses traditional machine learning models including MLP, XGBoost, and RF that process feature vectors directly without modeling relational structures. The second group consists of general GNNs including GCN, Skip-GCN, GraphSAGE, and GAT, which leverage message-passing mechanisms to capture node interactions. The third group includes specialized approaches such as PC-GNN, Care-GNN, MLaD^2, and DGT + XGBoost that incorporate advanced techniques such as selective neighbor aggregation strategies, decoupling training mechanisms, and hybrid pipeline architectures to address specific challenges in graph-based anomaly detection. This categorization enables systematic comparison across different architectural paradigms for graph-based anomaly detection.

C. Metrics and Implementation

To evaluate the performance of AML detection models, we employ three standard evaluation metrics: 1) the area under the receiver operating characteristic curve (AUC), which assesses the model's discriminative ability across all classification thresholds; 2) the average precision (AP), which summarizes the precision-recall performance without requiring threshold selection and is particularly suited for imbalanced datasets common in money laundering detection; and 3) the binary F1-score (F1), which evaluates the balance between precision and recall at the default classification threshold. These metrics collectively provide a comprehensive assessment: AUC evaluates discriminative performance across thresholds, AP reflects model performance under class imbalance conditions, and F1 measures practical classification effectiveness at the deployment threshold.

For baseline methods, we adopt the hyperparameters provided in their original articles when available. In cases where hyperparameters are not specified, we conduct a grid search over the following ranges: learning rates in $\{0.01, 0.05, 0.001\}$, dropout ratios in $\{0.3, 0.4, 0.5, 0.6\}$, weight decay coefficients in $\{10^{-3}, 10^{-4}, 10^{-5}\}$, and hidden dimensions in $\{16, 32, 64, 128, 256\}$, selecting appropriate configurations based on validation set. For our proposed DC-LCG model, we set the embedding dimension to 256 for both Elliptic and RD datasets, while adopting a 64-dimensional embedding for the T-Finance dataset to balance model complexity with dataset size. Across all experiments, we maintain a consistent batch size of 1024 and train for a maximum of 1000 epochs with an early stopping patience of 100 epochs on the validation loss.

Regarding data partitioning, we employ a 7:3 train-validation split ratio for the T-Finance dataset using scikit-learn's data partitioning utilities. For both Elliptic and RD datasets, considering their temporal characteristics, we use first 35 time steps

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
CLASSIFICATION PERFORMANCE (%) COMPARISON BETWEEN DC-LCG AND BASELINE METHODS

Dataset		Elliptic			RD			T-Finance		
Metric		F1	AUC	AP	F1	AUC	AP	F1	AUC	AP
Non-GNN Baseline	MLP	64.83±0.52	84.28±0.34	63.0±0.58	60.95±0.69	76.91±0.43	57.4±0.78	71.8±0.61	86.95±0.28	74.45±0.67
	Xgboost	78.85±0.41	91.51±0.26	76.58±0.48	70.97±0.55	85.82±0.34	68.92±0.64	78.27±0.45	92.8±0.23	83.18±0.52
	RF	76.68±0.38	89.49±0.29	75.06±0.44	69.01±0.51	83.91±0.37	67.55±0.59	77.02±0.42	91.68±0.25	81.55±0.49
GNNs (General)	GCN	66.7±0.87	86.9±0.56	64.5±0.94	61.03±1.02	80.45±0.71	58.87±1.15	71.4±0.79	89.8±0.42	74.0±0.89
	Skip-GCN	72.5±0.63	87.3±0.41	71.7±0.72	65.25±0.79	82.83±0.52	64.53±0.89	78.6±0.58	92.5±0.31	82.6±0.66
	GraphSAGE	69.7±0.74	86.1±0.48	67.4±0.81	62.73±0.91	80.75±0.61	60.66±0.98	76.4±0.65	91.3±0.33	79.9±0.74
	GAT	68.0±0.92	86.7±0.59	63.8±1.05	61.2±1.08	80.37±0.74	57.42±1.23	76.2±0.83	90.9±0.47	78.0±0.95
GNNs (Illicit Detection)	PC-GNN	74.76±0.47	87.23±0.33	72.89±0.54	69.08±0.61	82.67±0.42	67.4±0.68	80.14±0.43	92.67±0.28	82.23±0.51
	Care-GNN	74.32±0.51	87.84±0.36	72.12±0.58	68.69±0.65	82.3±0.45	66.71±0.72	81.84±0.46	92.45±0.31	83.67±0.54
	MLaD^2	76.13±0.44	88.45±0.31	74.34±0.51	72.12±0.57	83.83±0.39	69.51±0.63	83.41±0.39	93.23±0.26	85.45±0.47
	DGT + XGBoost	81.34±0.35	92.12±0.27	79.23±0.42	74.91±0.46	86.0±0.34	72.11±0.53	85.1±0.32	95.78±0.22	87.12±0.38
Ours	**DC-LCG**	**92.0±0.23**	**94.3±0.18**	**91.52±0.29**	**85.45±0.31**	**89.86±0.22**	**83.34±0.38**	**89.97±0.26**	**97.01±0.16**	**91.59±0.32**

Note: Bold entries indicate the best performance among all compared methods for each evaluation metric.

as the training set, while remaining time steps serve as the validation set. All experiments are implemented using PyTorch 1.12.1, with CUDA 11.2 and Python 3.8, and are conducted on a computing infrastructure equipped with four NVIDIA Tesla V100-SXM2 GPUs, each with 32 GB memory.

D. Quantitative Performance Comparison

Our comprehensive evaluation across ten repeated experiments for each method produces the average performance metrics with standard deviations presented in Table II. The results convincingly demonstrate that our proposed DC-LCG framework achieves superior performance across all evaluation metrics on all three benchmark datasets.

Among the non-GNN approaches, MLP exhibits relatively poor performance due to its inability to leverage the critical topological information inherent in transaction networks. Both XGBoost and RF demonstrate stronger results as tree-based ensemble methods. XGBoost effectively captures nonlinear relationships and complex feature interactions within transaction data through its gradient boosting framework. Similarly, Random Forest's ensemble methodology combines multiple decision trees via bagging, reducing variance and mitigating overfitting while handling high-dimensional feature spaces. Both methods derive their efficacy from processing heterogeneous node attributes, capturing diverse signals from transaction amounts, temporal patterns, and entity characteristics.

When examining general GNNs, we observe consistent performance limitations when processing financial transaction networks. GCN demonstrates modest performance, primarily constrained by its underlying homophily assumption, which poorly aligns with the heterophilic nature of money laundering networks. Skip-GCN shows significant improvements over GCN by mitigating over-smoothing issues, achieving the strongest performance among general GNNs across all three datasets. GraphSAGE demonstrates relatively decent performance across datasets, though its random neighborhood sampling strategy may occasionally miss certain critical transaction connections that are essential for money laundering detection. GAT achieves moderate performance compared with other general GNNs, as despite its attention mechanisms, it faces challenges in overcoming the homophily issues caused by illicit nodes that deliberately disguise their transaction patterns to mimic legitimate activities.

Specialized GNNs for illicit activity detection show significant performance improvements over general GNN models. PC-GNN and Care-GNN achieve nearly identical performance across the three datasets, both benefiting from their specialized designs for heterophilic financial networks. PC-GNN implements a neighbor selection mechanism to distinguish between different node types during message-passing. Care-GNN employs relation-specific aggregation to incorporate information from neighbors through an iterative procedure. While these architectures demonstrate effectiveness for financial transaction networks, their improvements remain limited. MLaD^2 employs a decoupling training approach with separate pre-training and fine-tuning phases, constructing transaction relationship networks based on node similarity to overcome traditional GNN limitations, achieving moderate performance improvements. DGT + XGBoost represents a hybrid approach combining dynamic graph transformers with gradient boosting, achieving the strongest baseline performance across all metrics and demonstrating the effectiveness of combining GNNs with ensemble learning methods. Despite acknowledging node heterogeneity, these specialized architectures remain limited by their focus on local neighborhoods. This local focus fails to capture the multihop transaction paths typical of money laundering operations.

Our DC-LCG framework addresses these limitations through effective modeling of both node heterogeneity and extended transaction sequences. Unlike previous approaches restricted to immediate neighbors, DC-LCG detects the long-range dependencies across multiple transaction paths critical for identifying money laundering activities. In comparison with the strongest baseline DGT + XGBoost, DC-LCG enhances performance metrics on the Elliptic dataset by 11%, 2%, and 5% for F1, AUC, and AP, respectively; on the RD by 14%, 14%, and 15%; and on the T-Finance by 4%, 1%, and 5%. The inclusion of

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

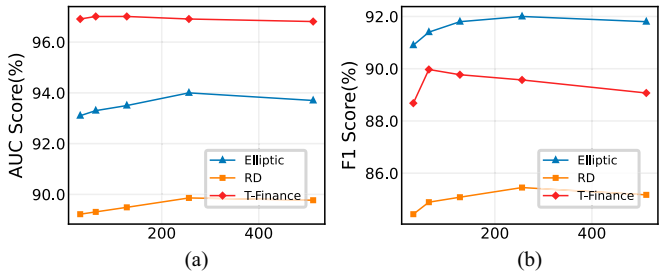


Fig. 3. Impact of hidden dimension on model performance across different datasets: (a) AUC scores; and (b) F1 scores.

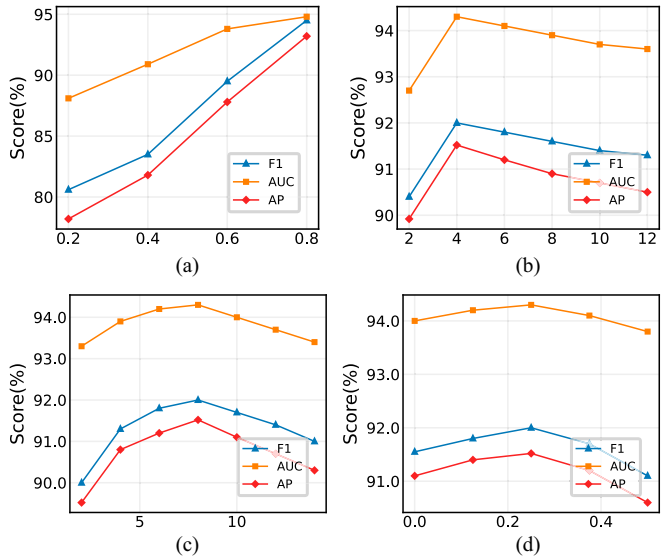


Fig. 4. Parameter sensitivity analysis of the DC-LCG on the elliptic dataset: (a) training ratio; (b) number of paths M; (c) path length L; and (d) dropout rate β.

standard deviations in our results demonstrates the statistical significance and reliability of these performance improvements across multiple experimental runs.

In conclusion, DC-LCG achieves state-of-the-art performance across all experimental benchmarks. This comprehensive comparative analysis validates that through its innovative mechanism of effectively integrating local and contextual features, DC-LCG significantly enhances the accuracy of AML detection.

E. Parameter Evaluation

In this section, we conduct a systematic investigation into parameter sensitivity of DC-LCG. First, to evaluate the sensitivity to hidden layer dimension, Fig. 3 shows the effect of increasing the hidden layer dimension from 32 to 512 on Elliptic, RD, and T-Finance datasets. While model performance remains generally stable, subtle performance fluctuations are still observed at specific dimensions. Specifically, the model achieves optimal performance on Elliptic, RD, and T-Finance with hidden layer dimensions of 256, 256, and 64, respectively. It is worth noting that the initial feature vector lengths for Elliptic, RD, and T-Finance are 166, 182, and 10, respectively. In general, selecting a hidden layer dimension appropriately higher than the initial feature length can effectively enhance the model's expressive power and generate more information-rich node embeddings.

We further explore the sensitivity of DC-LCG to other key parameters on the Elliptic dataset, including variations in the training set proportion, the number of random walk paths M, the path length L, and the dropout probability value β. These experiments further validate the model's adaptability and stability across various parameter settings. As shown in Fig. 4(a), DC-LCG maintains good performance when the training set proportion is adjusted from 20% to 80%, reflecting the model's robustness under different training data sizes. In Fig. 4(b), we increase M from 2 to 12, with model performance peaking at $M = 4$. This indicates that excessive random walk paths may introduce noise information, negatively impacting model performance. Fig. 4(c) illustrates the effect of increasing L from 2 to 14, with peak performance achieved at $L = 8$. This suggests that an appropriate path length effectively captures contextual information, while longer paths may lead to information redundancy and affect the model's detection performance. As depicted in Fig. 4(d), we vary β from 0 to 0.5, with optimal performance reached at $\beta = 0.25$. This demonstrates that proper node removal can reduce the influence of noise.

In summary, the experimental results demonstrate that our model exhibits good stability under various parameter settings.

F. Ablation Study

To evaluate the effectiveness of the internal components in DC-LCG, we conducted ablation studies to assess the performance contributions of three core modules: the dynamic grouping and aggregation module (DMG), the dynamic path pruning and fusion module (DMP), and the mutual attention fusion module (MAF). All experiments were conducted based on enhanced node features processed by the dynamic value field encoder. Additionally, we performed comprehensive ablation studies on the internal components of each module to thoroughly examine their individual contributions, as detailed in Table III. In our ablation analysis, 'w/o' denotes experiments conducted without the specified component.

Within the DMG module, intralayer aggregation produces more substantial performance impacts than interlayer aggregation, with F1 decreases of 8% versus 6% on Elliptic when removed, respectively. This indicates that probability-weighted aggregation within separated groups plays a more crucial role in handling heterogeneous connectivity patterns by ensuring nodes with similar behavioral characteristics reinforce each other during feature learning. The interlayer mechanisms provide complementary cross-group information exchange that contributes essential discriminative power for comprehensive heterogeneity mitigation.

For the DMP module, multipath semantic fusion serves as the more critical component than dynamic selective pruning, with removal causing F1 drops of 8% versus 4% on Elliptic. This validates that multipath semantic fusion provides the primary capability for capturing complex multihop transaction

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPREHENSIVE ABLATION STUDY ON MODEL COMPONENTS (%)

Component	Elliptic			T-Finance		
	F1	AUC	AP	F1	AUC	AP
DMG only	87.55	93.01	86.37	86.21	97.12	87.51
w/o intra	84.23	90.76	82.12	83.89	94.45	84.23
w/o inter	86.12	92.45	85.56	85.12	96.78	86.67
DMP only	88.22	93.69	87.04	85.66	96.49	86.23
w/o pruning	87.45	92.12	86.23	84.89	96.12	87.78
w/o multi-path	85.89	91.34	84.67	83.34	95.21	86.11
Full model	92.00	94.30	91.52	89.97	97.01	91.59
w/o attention	90.67	93.89	89.56	88.01	96.89	90.67
Concatenation	90.12	93.67	89.18	87.45	96.12	90.67

dependencies and contextual patterns through the aggregation of diverse information from multiple transaction paths. Meanwhile, selective pruning enhances downstream effectiveness by improving signal-to-noise ratios and enabling focused computational resources on the most relevant transaction patterns.

Analysis of the MAF module demonstrates that attention-based integration achieves 2% better F1 performance compared with simple concatenation on Ellipti, and 1.3% improvement over the attention-free approach. These consistent improvements across all evaluation metrics indicate that the mutual attention mechanism provides systematic benefits in capturing complementary relationships between local neighborhood characteristics and contextual transaction sequences.

Most notably, when DMP, DMG, and MAF work in combination as integral components of the complete DC-LCG model, we observe substantial performance improvements across all evaluation metrics. Compared with employing individual modules, the complete DC-LCG model exhibits remarkable performance improvements, with F1 and AP scores increasing by 3%–7%. These empirical findings substantiate that our integrated approach successfully addresses the two primary challenges in AML detection. The DMP component effectively captures and processes features from multihop money laundering paths, enabling the identification of complex transaction patterns. Concurrently, the DMG component specializes in detecting locally disguised nodes within transaction networks. The MAF component ensures optimal integration of information from both modules through attention-based mechanisms. Through their complementary operational mechanisms, these three modules function synergistically to significantly enhance the overall detection performance of the model.

G. Computational Complexity

To evaluate DC-LCG's computational complexity, we compare computational complexity and runtime performance against the best-performing baseline DGT + XGBoost. Since this baseline employs two-stage training with the primary computational bottleneck in the deep learning stage, we focus our analysis on the DGT component.

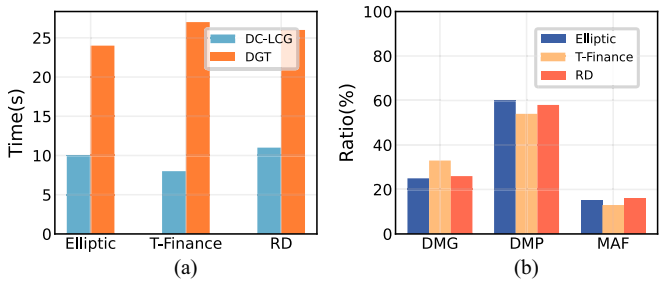


Fig. 5. Training time and module analysis across different datasets: (a) training time comparison between DC-LGC and DGT methods; and (b) DC-LGC module ratio distribution.

In conclusion, our analysis demonstrates that DC-LCG achieves competitive computational complexity across experimental benchmarks. This efficiency analysis validates that through its architectural design, DC-LCG maintains reasonable computational efficiency while achieving effective detection performance.

We first analyze the theoretical computational complexity of DC-LCG's architecture. DC-LCG's computational complexity distributes across three core modules. The DMG requires $\mathcal{O}(|E| + N \times S \times d_h)$ operations, the DMP has complexity $\mathcal{O}(M \times T \times N + M \times T^2 \times d_h \times L_{\text{tm}})$, and the MAF requires $\mathcal{O}(d_h^2)$ operations. Our DC-LCG model's primary computational bottleneck resides in the DMP. Due to the pruning mechanism's effective control of both path length and path quantity, the transformer encoder's computational complexity remains limited, making random walk generation the dominant computational component with complexity $\mathcal{O}(M \times T \times N)$. In comparison, the most computationally intensive components of the baseline DGT model require $\mathcal{O}(|V|^3)$ matrix precomputation for diffusion similarities and $\mathcal{O}(\tau \times |E| \times \omega \times |V| \times \kappa)$ neighbor searching operations. Our approach's linear complexity demonstrates superior computational efficiency compared with the baseline's cubic operations.

We subsequently conduct empirical per-epoch runtime experiments to validate the theoretical analysis. Fig. 5(a) demonstrates the runtime efficiency comparison, showing consistent efficiency advantages across all datasets, with DC-LCG achieving substantial speedups ranging from 2.4× to 3.4× compared with the baseline, with the most significant improvement observed on T-Finance dataset. Fig. 5(b) provides detailed insight into the computational distribution within DC-LCG's architecture across the three datasets. The analysis reveals that the DMP consistently represents the largest computational component, followed by the DMG as the second-largest component, while MAF requires the least computational resources. This consistent distribution pattern across different datasets validates our theoretical complexity analysis.

H. Case Study

To validate our claim that DC-LCG generates more discriminative node embeddings, we conduct a case study to showcase the differences between embeddings generated by some baseline models and DC-LCG on the Elliptic dataset. We employ

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

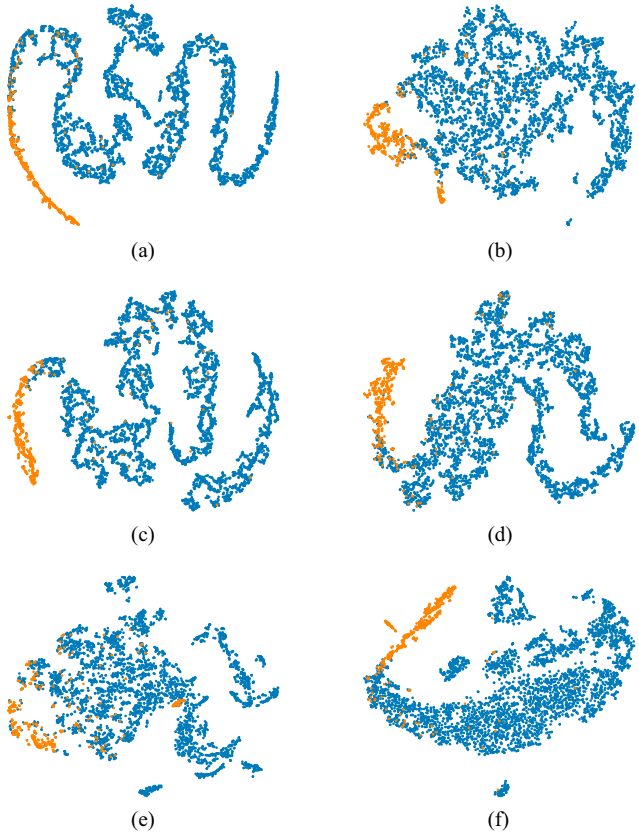


Fig. 6. Visualization of node embeddings on the elliptic dataset using t-SNE dimensionality reduction: (a) GCN; (b) GAT; (c) GraphSAGE; (d) Skip-GCN; (e) Care-GNN; and (f) DC-LCG. The red and blue nodes represent illicit and licit nodes respectively.

t-SNE to map node embeddings into a two-dimensional space, and the resulting embeddings are visualized in Fig. 6.

For GCN, the embeddings exhibit less distinct separation between illicit and licit node clusters, with observable class overlap shown in Fig. 6(a). This limited separation reflects GCN's fundamental constraints in handling heterophilic financial networks where illicit nodes deliberately mimic legitimate patterns. GAT demonstrates marginal improvement in Fig. 6(b), yet still shows certain limitations in distinguishing between illicit and licit nodes, with indistinct class boundaries persisting despite its attention mechanisms.

GraphSAGE generates embeddings with reduced interclass mixing compared with previous methods, though some boundary ambiguity remains evident in Fig. 6(c). The improved separation potentially reflects benefits of its sampling-based approach, while remaining limitations may relate to its random neighborhood sampling strategy. Skip-GCN achieves improved embedding quality with clearer class boundaries and more cohesive cluster formation in Fig. 6(d), indicating that addressing over-smoothing issues contributes to enhanced discriminative capability. However, the method still demonstrates limitations in capturing long-range transaction dependencies that are crucial for comprehensive money laundering detection.

Care-GNN produces well-defined clusters with improved separation in Fig. 6(e), demonstrating the benefits of specialized architectural design for financial network applications. Nevertheless, its focus on local neighborhood relationships constrains its ability to model extended transaction sequences, resulting in suboptimal performance for complex money laundering patterns. In contrast, DC-LCG demonstrates enhanced boundary separation and forms more cohesive clusters in Fig. 6(f). DC-LCG generates clear boundaries with increased separation between different node categories, forming distinct clusters with reduced interclass overlap and demonstrating improved discriminative capability compared with baseline approaches. The enhanced embedding quality reflects DC-LCG's dual-context mechanism that effectively captures both local neighborhood characteristics and extended transaction sequence patterns.

These observations further validate the effectiveness of DC-LCG in generating highly discriminative node embeddings and its superiority over GCN in AML detection.

VI. Conclusion

In this article, we propose the DC-LCG framework, which constructs an effective method for identifying money laundering transactions through the collaborative design of local and context views. The local view extracts node neighborhood features to address the camouflaging characteristics of money laundering nodes, while the context view extracts multihop transaction path patterns to capture illicit fund flows within transaction networks. Experimental results demonstrate that DC-LCG significantly outperforms existing methods on multiple datasets, validating its effectiveness in detecting money laundering activities in transaction networks. Our research provides novel insights and practical references for money laundering chain modeling and detection of money laundering nodes. DC-LCG demonstrates competitive performance. In future work, we will design more efficient path sampling strategies and distributed computing approaches to address the computational overhead from generating multiple random walk paths. Additionally, we will further enhance model interpretability through attention visualization mechanisms and path-based explanations, which could provide better insights into the decision-making process.

References

[1] D.-A. Nedelcu, "Money laundering by means of offshore companies," *Analele Universităţii Titu Maiorescu*, vol. 18, no. XVIII, pp. 173–185, 2019.

[2] A. S. M. Irwin, J. Slay, K.-K. Raymond Choo, and L. Lui, "Money laundering and terrorism financing in virtual environments: a feasibility study," *J. Money Laundering Control*, vol. 17, no. 1, pp. 50–75, 2014.

[3] V. Dyntu and O. Dykyi, "Cryptocurrency in the system of money laundering," *Baltic J. Econ. Stud.*, vol. 4, no. 5, pp. 75–81, 2018.

[4] X. Tan, T.-F. T. Tse, S. M. Yiu, and H.-M. H. Lam, "A case study on multi-countries money laundering scheme and a proposed automatic detection system," in *Proc. ACI ICCWS*, Johannesburg, South Africa, Mar. 2024, pp. 385–394.

[5] A. N. Susanto and W. Afifah, "The role of institutions supporting the investigation of evidence in money laundering crimes using cryptocurrency," *Innovative: J. Social Sci. Res.*, vol. 4, no. 6, pp. 3144–3160, 2024.

[6] Y. Yang, B. Lian, L. Li, C. Chen, and P. Li, "Dbscan clustering algorithm applied to identify suspicious financial transactions," in *Proc. IEEE CyberC,* Shanghai, China, Oct. 2014, pp. 60–65.

[7] Z. Chen, A. Nazir, E. N. Teoh, E. K. Karupiah, et al., "Exploration of the effectiveness of expectation maximization algorithm for suspicious

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.

transaction detection in anti-money laundering," in *Proc. IEEE ICOS*, Makassar, Indonesia, Oct. 2014, pp. 145–149.
[8] W.-Y. Chen, S.-H. Li, and Y.-H. Wang, "Research on natural language processing in financial risk detection," in *Proc. Springer IC3*, Kyoto, Japan, Sep. 2020, pp. 448–455.
[9] J. Tang and J. Yin, "Developing an intelligent data discriminating system of anti-money laundering based on SVM," in *Proc. IEEE ICMLC*, Guangzhou, China, Aug. 2005, pp. 3453–3457.
[10] L.-T. Lv, N. Ji, and J.-L. Zhang, "A RBF neural network model for anti-money laundering," in *Proc. IEEE ICWAPR*, Hong Kong, China, Aug. 2008, pp. 209–215.
[11] Gao, Z. "Application of cluster-based local outlier factor algorithm in anti-money laundering," in *Proc. IEEE MASS*, Macau, China, Oct. 2009, pp. 1–4.
[12] V. Jayasree and R. S. Balan, "Money laundering regulatory risk evaluation using bitmap index-based decision tree," *J. Assoc. Arab Univ. Basic Appl. Sci.*, vol. 23, pp. 96–102, 2017.
[13] M. A. Harlev, H. S. Yin, K. C. Langenheldt, R. R. Mukkamala, and R. Vatrapu, "Breaking bad: De-anonymising entity types on the bitcoin blockchain using supervised machine learning," in *Proc. HICSS*, Waikoloa Village, USA, Jan. 2018, pp. 3497–3506.
[14] Y. Hu, S. Seneviratne, K. Thilakarathna, K. Fukuda, and A. Seneviratne, "Characterizing and detecting money laundering activities on the bitcoin network," 2019, vol. 23, *arXiv:1912.12060*.
[15] A. Kumar, S. Das, and V. Tyagi, "Anti money laundering detection using Naïve Bayes classifier," in *Proc. IEEE GUCON*, Noida, India, Oct. 2020, pp. 568–572.
[16] P. Zhao, W. Tian, L. Xiao, X. Liu, and J. Wu, "An attention-based long short-term memory framework for detection of bitcoin scams," in *Proc. IEEE HDIS*, Tianjin, China, Dec. 2022, pp. 21–26.
[17] X. Sun, T. Yang, and B. Hu, "LSTM-TC: Bitcoin coin mixing detection method with a high recall," *Appl. Intell.*, vol. 52, no. 1, pp. 780–793, 2022.
[18] J. F. Rodríguez et al., "A natural language processing approach for financial fraud detection," in *Proc. CEUR-WS. org*, Rome, Italy, Jun. 2022, pp. 135–149.
[19] Q. Umer, J.-W. Li, M. R. Ashraf, R. N. Bashir, and H. Ghous, "Ensemble deep learning-based prediction of fraudulent cryptocurrency transactions," *IEEE Access*, vol. 11, pp. 95213–95224, 2023.
[20] M. P. Tatulli, T. Paladini, M. D'Onghia, M. Carminati, and S. Zanero, "Hamlet: A transformer based approach for money laundering detection," in *Proc. Springer CSCML*, Be'er Sheva, Israel, Jun. 2023, pp. 234–250.
[21] S. Huang, Y. Xiong, Y. Xie, T. Qiu, and G. Wang, "Robust sequence-based self-supervised representation learning for anti-money laundering," in *Proc. ACM CIKM*, Boise, USA, Oct. 2024, pp. 4571–4578.
[22] K. Girish and B. Bhowmik, "Money laundering detection in banking transactions using RNNs and hybrid ensemble," in *Proc. IEEE ICCCNT*, Mandi, India, Jun. 2024, pp. 1–7.
[23] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, Toulon, France, Apr. 2017.
[24] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, Vancouver, Canada, Apr. 2018.
[25] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NIPS*, Long Beach, USA, Dec. 2017, pp. 1024–1034.
[26] M. Guang, C. Yan, Y. Xu, J. Wang, and C. Jiang, "Graph convolutional networks with adaptive neighborhood awareness," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 11, pp. 7392–7404, Nov. 2024.
[27] Y. Liu et al., "Pick and choose: A GNN-based imbalanced learning approach for fraud detection," in *Proc. ACM / IW3C2 WWW*, Ljubljana, Slovenia, Apr. 2021, pp. 3168–3177.
[28] Y. Yu, Y. Xu, J. Wang, Z. Li, and B. Cao, "Anti-money laundering risk identification of financial institutions based on aspect-level graph neural networks," in *Proc. IEEE QRS-C*, Guangzhou, China, Dec. 2022, pp. 542–546.
[29] Z. Chai et al., "Towards learning to discover money laundering sub-network in massive transaction network," in *Proc. AAAI*, Washington, USA, Feb. 2023, pp. 14 153–14 160.
[30] W. Hyun, J. Lee, and B. Suh, "Anti-money laundering in cryptocurrency via multi-relational graph neural network," in *Proc. Springer PAKDD*, Osaka, Japan, May 2023, pp. 118–130.
[31] X. Li, Y. Li, X. Mo, H. Xiao, Y. Shen, and L. Chen, "DIGA: Guided diffusion model for graph recovery in anti-money laundering," in *Proc. AMC KDD*, Long Beach, USA, Aug. 2023, pp. 4404–4413.
[32] S. Jing, L. Chen, Q. Li, and D. Wu, "DoS-GNN: Dual-feature aggregations with over-sampling for class-imbalanced fraud detection on graphs," in *Proc. IEEE IJCNN*, Yokohama, Japan, Jun. 2024, pp. 1–8.
[33] M. R. Karim, F. Hermsen, S. A. Chala, P. De Perthuis, and A. Mandal, "Scalable semi-supervised graph learning techniques for anti money laundering," *IEEE Access*, vol. 12, pp. 50012–50029, 2024.
[34] X. Luo, X. Han, W. Zuo, X. Wu, and W. Liu, "Mlad^2: A semi-supervised money laundering detection framework based on decoupling training," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 4518–4533, 2024.
[35] M. Guang, C. Yan, G. Liu, J. Wang, and C. Jiang, "A novel neighborhood-weighted sampling method for imbalanced datasets," *Chin. J. Electronics*, vol. 31, no. 5, pp. 969–979, 2022.
[36] M. Weber et al., "Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics," in *Proc. ACM KDD*, Anchorage, USA, Aug. 2019.
[37] M. Guang, C. Yan, Y. Xu, J. Wang, and C. Jiang, "A multichannel convolutional decoding network for graph classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13 206–13 216, Oct. 2024.
[38] Q. Wang, W.-T. Tsai, and T. Shi, "GraphALM: Active learning for detecting money laundering transactions on blockchain networks," *IEEE Netw.*, vol. 39, no. 2, pp. 294–303, Mar. 2025.
[39] J. Tang, J. Li, Z. Gao, and J. Li, "Rethinking graph neural networks for anomaly detection," in *Proc. PMLR ICML*, Baltimore, USA, Jul. 2022, pp. 21076–21089.
[40] G. E. Hinton, "Connectionist learning procedures," in *Machine Learning*, Morgan Kaufmann, 1990, pp. 555–610.
[41] M. A. Harlev, H. Sun Yin, K. C. Langenheldt, R. Mukkamala, and R. Vatrapu, "Breaking bad: De-anonymising entity types on the bitcoin blockchain using supervised machine learning," in *Proc. HICSS*, Waikoloa Village, USA, Jan. 2018, pp. 3497–3506.
[42] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
[43] Y. Dou, Z. Liu, L. Sun, et al., "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proc. ACM CIKM*, Galway, Ireland, Oct. 2020, pp. 315–324.



Zhong Li received the Ph.D. degree in computer software and theory from Tongji University, Shanghai, China, in 2015.

She is currently a Professor with Donghua University, Shanghai. Her research interests include transaction fraud detection, federated learning, privacy-preserving computation, and network security.



Xinyu Yin (Student Member, IEEE) is currently working toward the M.S. degree in electronic information with the School of Information and Intelligent Science, Donghua University, Shanghai, China.

Her research interest includes security and privacy issues in financial networks.

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.



Mingjian Guang (Member, IEEE) received the Ph.D. degree in computer science and technology from Tongji University, Shanghai, China, in 2024.

He is currently a Lecturer with Donghua University, Shanghai, China. His research interests include GNNs, deep learning, ensemble learning, and fraud detection. He has authored or coauthored multiple research works on top-tier conferences and journals, including IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, *Information Sciences*, and IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING.



Changjun Jiang received the Ph.D. degree in control theory and engineering from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 1995.

He is currently a Professor with the Department of Computer Science and Engineering, Tongji University, Shanghai, China. He has been elected to Chinese Academy of Engineering in 2021. His research interests include concurrent theory, petri net, formal verification of software, concurrency processing, and intelligent transportation systems. He is the Council Member of China Automation Federation and Artificial Intelligence Federation, the Vice-Director of Professional Committee of Petri Net of the China Computer Federation, the Vice-Director of Professional Committee of Management Systems of the China Automation Federation, and Information Area Specialist of Shanghai Municipal Government.

Authorized licensed use limited to: Dayananda Sagar College of Engineering. Downloaded on August 08,2026 at 03:49:06 UTC from IEEE Xplore. Restrictions apply.